

Deep learning for postprocessing tutorial

2026/06/23

version 2026.06.01

Ruben Sanchez-Garcia

rsanchezgarcia@faculty.ie.edu

1. Introduction

Cryo-EM maps often suffer from a loss of contrast at high frequencies, hindering interpretation. To facilitate modelling, maps undergo post-processing to enhance contrast at medium to high resolution frequencies. However, traditional B-factor-based methods amplify both signal and noise, while applying a single B-factor across the entire map ignores local quality variations, resulting in inconsistent sharpening.

DeepEMhancer offers an alternative approach for local post-processing. Trained on pairs of experimental and model-enhanced maps, it selectively improves signal-to-noise ratio and visualization automatically.

This tutorial provides comprehensive guidance on downloading, installing, and running DeepEMhancer. We'll demonstrate its usage as a standalone command-line tool, a Scipion protocol, and a CryoSPARC wrapper. We'll also address potential failure modes and discuss complementary map post-processing techniques that can provide alternative perspectives on your data. Additionally, we'll explore anisotropy attenuation capabilities through both DeepEMhancer and splsoNet and introduce some of the new features in LocScale2.

2. DeepEMhancer Installation (10 mins)

For standalone usage and also for the CryoSPARC wrapper, DeepEMhancer needs to be installed using conda or pip. While it is possible to manually install it to accommodate it to some predefined dependencies, we recommend creating a new fresh environment for DeepEMhancer. The following recipe should work in most situations:

1. Clone the GitHub repository and `cd` inside

```
git clone https://github.com/rsanchezgarc/deepEMhancer
cd deepEMhancer
```

2. Create a conda environment with the required dependencies. You can give any name to the environment with the `-n` flag. Here, we have arbitrarily chosen "deepEMhancer_env".

```
conda env create -f deepEMhancer_env.yml -n deepEMhancer_env
```

Conda can be slow, so you can use mamba instead.

```
mamba env create -f deepEMhancer_env.yml -n deepEMhancer_env
```

3. Activate the environment. You always need to activate the environment before executing the program.

```
conda activate deepEMhancer_env
```

4. Set the `LD_LIBRARY_PATH`.

```
conda env config vars set
LD_LIBRARY_PATH=${CONDA_PREFIX}/lib/python3.9/site-
packages/nvidia/cudnn/lib:${CONDA_PREFIX}/lib/:$LD_LIBRARY_PATH
conda activate deepEMhancer_env #To make the changes effective
```

5. Install deepEMhancer

```
python -m pip install . --no-deps
```

Notice the “.”. It means that we are installing the package from the current folder source code (that we just downloaded). We use `--no-deps` to skip installing any further dependencies, as they have been previously installed with conda or mamba.

6. Download our deep learning models

```
deepemhancer --download
```

By default, this downloads the models to the `$HOME/.local/share/deepEMhancerModels` folder. If you want to store them somewhere else, you can just use

```
deepemhancer --download /my/preferred/directory
```

7. Ready! Do not forget to activate the environment for future usages. For a complete description of the parameters and options use:

```
deepemhancer -h
```

Optionally, you can remove the folder, since the command `deepemhancer` will be available anywhere once you activate the environment. Now you should be able to run DeepEMhancer using the command line.

DeepEMhancer can also be automatically installed from anaconda cloud as

```
conda install deepEMhancer -c rsanchez1369 -c anaconda -c conda-forge
```

GUI options:

- DeepEMhancer can be automatically installed in Scipion. Install Xmipp’s deepLearningToolkit on the plugging manager or using the following command:

```
scipion installb deepLearningToolkit
```

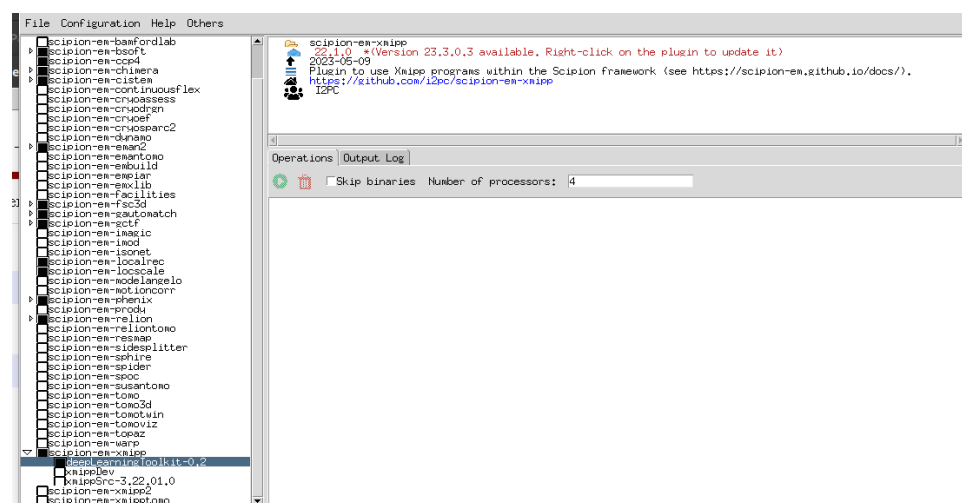


Figure 1. Installing DeepEMhancer in Scipion. DeepEMhancer can be easily installed as part of Xmipp’s deepLearningToolkit by selecting it from the right-hand side menu and clicking on the green arrow.

- CryoSPARC wrapper:

You first need a working installation of the deepEMhancer command line program. Install it as explained at the beginning of this section. After that, please, follow the instructions at <https://guide.cryosparc.com/processing-data/all-job-types-in-cryosparc/post-processing/job-deepemhancer-wrapper>.

3. Collecting the required data (5 mins)

In this tutorial, we are going to use three different EMDB entries, although you are more than welcome to run the tutorial with your own maps. Please download the half-maps (you can use `wget` for that) and the published volume for each of them. You also might want to download the atomic models to compare them with the maps, and to run `locScale2`.

- EMD-7099. A multidrug resistance protein (MRP1) with 17 transmembrane helices in which the signal coming from the lipids makes it difficult to visualize several regions of the protein.
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-7099/map/emd_7099.map.gz
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-7099/other/emd_7099_half_map_1.map.gz
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-7099/map/emd_7099.map.gz
 - <https://files.rcsb.org/download/6BHU.cif>
- EMD-8731. The influenza hemagglutinin (HA) trimer reconstructed from tilted micrographs that exhibit moderate amounts of anisotropy.
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-8731/map/emd_8731.map.gz
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-8731/other/emd_8731_half_map_1.map.gz
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-8731/other/emd_8731_half_map_2.map.gz
 - <https://files.rcsb.org/download/7VDF.cif> (This is the PDB for another HA structure)
- EMD-15673. A TasA fibre protein in which automatic DeepEMhancer normalization fails.
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-15673/map/emd_15673.map.gz
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-15673/other/emd_15673_half_map_1.map.gz
 - ftp://ftp.ebi.ac.uk/pub/databases/emdb/structures/EMD-15673/other/emd_15673_half_map_2.map.gz

Open them in Chimera/ChimeraX and try to identify some of the problematic regions.

Note: You have downloaded .gz compressed files. You might need to decompress them before using Chimera. Use `gzip -d FILENAME` to do so.

4. DeepEMhancer usage

inputs:

The input(s) required to run DeepEMhancer are either the **half maps** (`-i` and `-i2`) or the average **raw map** (`-i`). Although the result will be the same if you decide to use half maps or a non-sharpened non-masked map, choosing the half maps as inputs can help prevent unintentional use of other post-processed maps. Pay special attention to the noise around the protein. There should be at least a thick shell of noise around the protein, or even better, the whole cube should be filled with noise.

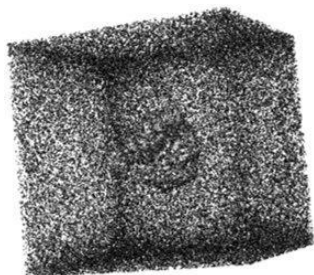


Figure 2. Input maps need to contain noise around the protein. Non-sharpened non-masked maps, with noise around the protein, are the only valid input for DeepEMhancer.

Since DeepEMhancer was trained on half-maps, we cannot know how it will behave if post-processed maps are provided as input instead. Thus, you should never be using them as input.

We can start executing DeepEMhancer on one of the examples.

```
conda activate deepEMhancer_env #Activate the environment
deepemhancer -i emd_7099_half_map_1.map -i2 emd_7099_half_map_2.map -o emd_7099_demh.map
```

This will generate a post-processed map named “emd_7099_demh.map” from the two provided half-maps.

By default, if no arguments are provided, `deepemhancer` will run on your `CUDA_VISIBLE_DEVICE=0`.

NOTE: Depending on your hardware and software versions you may need to set the following environmental variable `TF_FORCE_GPU_ALLOW_GROWTH='true'`. If you see some strange error mentioning CUDA, CUDNN, Conv3d or Conv3d_transpose, you probably are in one of those cases.

You can export it in your shell by

```
export TF_FORCE_GPU_ALLOW_GROWTH='true'
```

or prepend it to the `deepemhancer` command

```
TF_FORCE_GPU_ALLOW_GROWTH='true' deepemhancer -i emd_7099_half_map_1.map -i2 emd_7099_half_map_2.map -o emd_7099_demh.map
```

parameters:

DeepEMhancer should run fully automatic. Still, there are a couple of parameters that may modify the generated output as well as a few others that do not impact the quality of the results but have an effect on the execution time and hardware requirements.

- Parameters that **DO affect** the output:
 - **Model power (-p):** Deep learning model to use, three options are available:
 - **tight target (-p tightTarget):** The default model. Trained on tightly masked maps with resolutions from 2.8 to 6 Å. The training set includes cubes with protein and a fair number of empty cubes as well. This is the recommended model for most cases.
 - **highRes (-p highRes):** This model was trained on maps with resolution better than 4 Å only and fewer empty cubes. It is the recommended choice for high resolution maps. Avoid using it if the map quality is quite heterogeneous. Notice that this model is less effective than the tight target model suppressing noise. This is the price we need to pay to get sharper maps.
 - **wide target (-p wideTarget):** This is the most conservative model. We recommend it when you have areas in which the signal and noise are at almost identical contour values. Whereas tightTarget and highRes might delete those regions considering them only noise, the wide target model will probably preserve them. Of course, the results will not be as sharp as in the other cases.

NOTE: In case your map shows high heterogeneity, with parts of high resolution and low-resolution areas, using both tightTarget and highRes models and studying both maps simultaneously and manually deciding which areas are better sharpened by each model is recommendable.

Now it is a good time to try the different normalization modes. If you execute them on the EMD-7099 you can get a good feeling of how distinct they are.

```
deepemhancer -i emd_7099_half_map_1.map -i2 emd_7099_half_map_2.map -o emd_7099_demh.map -p highRes
```

```
deepemhancer -i emd_7099_half_map_1.map -i2 emd_7099_half_map_2.map -o emd_7099_demh.map -p wideTarget
```

Bonus question. Do you think you can trace residues chain A residues 195 to 202 (SPLFSGTI) PDBID: [6bhu](#)?

- **Input normalization:** Properly normalizing your input maps is a key requirement that, hopefully, is automatically carried out by DeepEMhancer using a robust heuristic. The program has a default normalization mode that can be run automatically or manually, and a mask-based normalization mode (discouraged).
 - **Default:** the default normalization mode computes the mean and the standard deviation of the noise around the protein (outer shell) and shifts

and rescales the map intensity so that the mean and standard deviation of the noise shell becomes 0 and 0.1 respectively. Note that this normalization mode requires your volume containing noise.

- **Automatic normalization** estimates noise and standard deviation computing radial profiles. While this works most of the time, it may fail for very long specimens (e.g., fibre proteins) or those having hollow structures (e.g., big viruses).

If everything goes as expected, you should read a message like “Automatic radial noise detected beyond 28 % of volume side”. In some other cases, it may warn you that the first trial failed, and it is trying to find the noise somewhere else. Warning messages include “The input might be a fibre, assuming no masking till radius 50 %” or “Automatic radial noise detection may have failed. No trend change found. Guessing radial noise of radius 50 %.” If the programs keep executing, there are high chances automatic normalization worked properly despite the warnings, but if you see that the output is not of enough quality, you probably want to try the next option, “Normalization from statistics”.

- **Normalization from statistics (`--noiseStats`)**. For those cases in which automatic normalization fails, DeepEMhancer will raise an error. You can circumvent it by manually estimating the noise statistics and providing them to DeepEMhancer. Manual estimation can be done in many different ways, but probably, the easiest one is to open your map in Chimera/ChimeraX, select a noise-only region, and then compute the statistics in that region.
- **Normalization from binary mask (`--binaryMask`)**. This option was introduced to allow DeepEMhancer to be executed on masked non-sharpened maps. The user needs to provide a tight binary mask of the protein (1 protein 0 background). This option is not compatible with any non-default model power options. In addition, the results tend to be of lower quality, so we discourage its use whenever possible.

Let's execute DeepEMhancer on one of those examples in which automatic normalization fails. The EMD-15673 is a fibre protein that spans the entire length of the cube. Consequently, radial profiles are not a feasible way to estimate the noise statistics and DeepEMhancer will abort execution. Try it yourself.

```
deepemhancer -i emd_15673_half_map_1.map -i2  
emd_15673_half_map_2.map -o emd_15673_demh_default.map
```

Hopefully, we can run DeepEMhancer on **Normalization from statistics** mode once we manually estimate the noise mean and standard deviation. In order to estimate them manually, you can open the input map in ChimeraX and use the following commands.

1. `volume copy #1`

This will create a copy of your volume

2. `volume #2 region 80,80,80,100,100,100`

This will crop a chunk of the map. You need to carefully select the coordinates so that you only crop a noise-only chunk. Trial and error may be required.

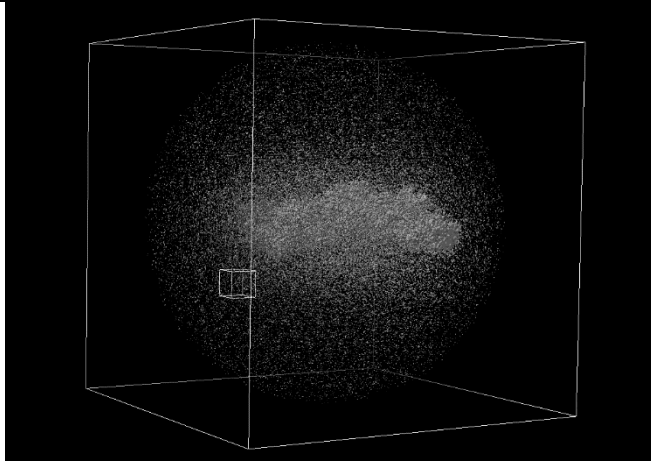


Figure 3. ChimeraX visualization of the map crop we have selected to estimate the map noise statistics.

3. `measure mapstats #2`

This reports the map crop statistics

Map emd_15673_half_map_1.map copy #3, subregion 80,80,80,100,100,100, minimum -0.01661, maximum 0.01477, mean -2.6e-05, SD 0.004104, RMS 0.004104

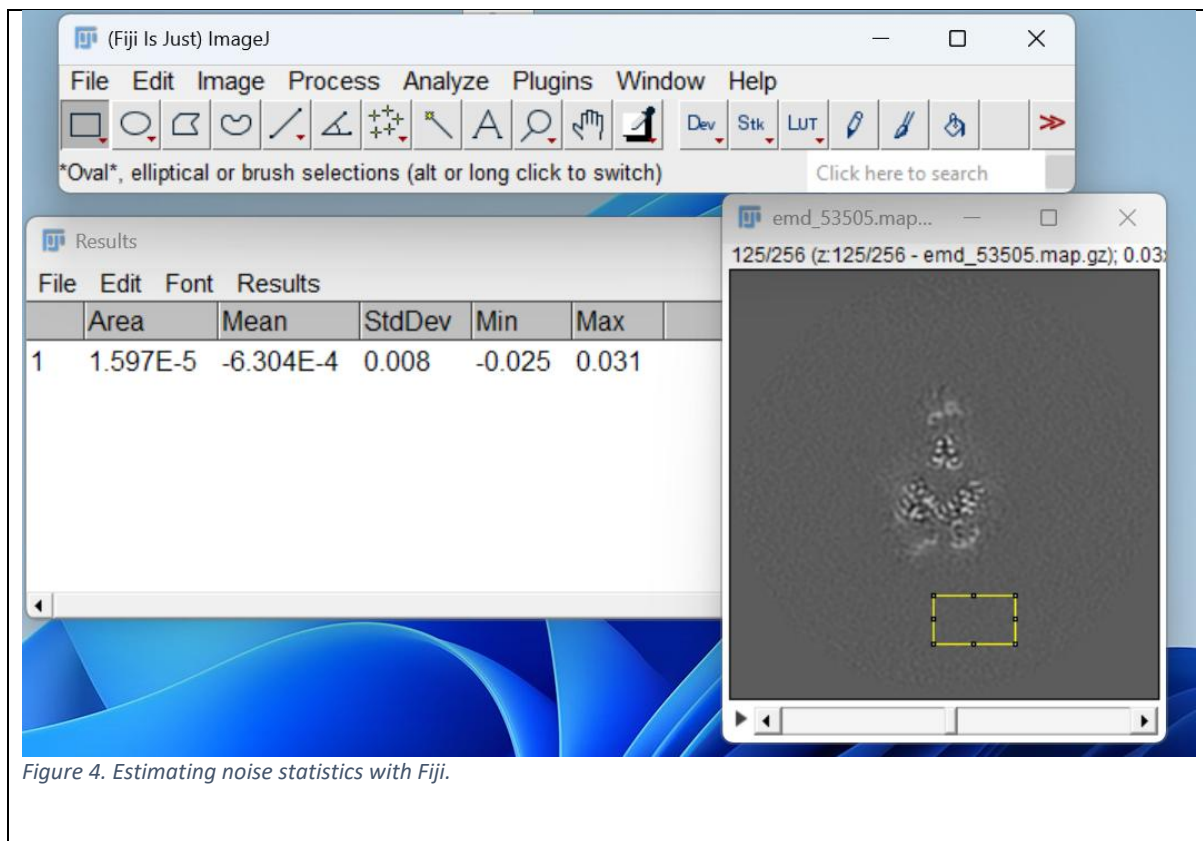
This command supports the optional argument `subregion i1,j1,k1,i2,j2,k2`, but you need to know where the subregion is, hence the need to do the copy.

Now you can run DeepEMhancer using the estimated statistics.

```
deepemhancer -i emd_15673_half_map_1.map -i2
emd_15673_half_map_2.map -o emd_15673_demh_default.map --
noiseStats -0.000026 0.004104
```

Note: If you estimate the statistics on one of the half-maps, you need to adjust the standard deviation dividing it by $\sqrt{2}$ (the standard deviation of the average map should be smaller than the standard deviation of the half maps). The average of the half maps and the average map should be the same.

An alternative to Chimera is to compute the statistics on 2D slices using a program like ImageJ/Fiji.



- **Remove small CC after processing (--cleaningStrength):** In addition to the deep post-processing, it is possible to run an additional final post-processing step in which the disconnected blobs are removed. This functionality is equivalent to “Hide dust in Chimera”. If you are going to use it, you need to provide as argument the size of the blobs to be removed as the fraction of the total volume of the box. This step can be quite computationally demanding, can remove some regions of interest and, in addition, it could be difficult to decide which is the desired size of the blobs to be removed. But it could be useful if you need a fully automatic solution within a pipeline. Our advice is not to use it if you are going to manually inspect the map after processing, since you can always use Hide dust later.

You can try the effect of --cleaningStrength. The EMD-7099 is a good candidate to try, although it will take some time. Try different values, for instance 0.001, 0.01 and 0.1.

```
deepemhancer -i emd_7099_half_map_1.map -i2
emd_7099_half_map_2.map -o emd_7099_demh_highRes_cc01.map -p
highRes --cleaningStrength 0.1
```

- **Parameters that DO NOT affect the output:**
 - **GPU IDs (-g):** By default, it is executed in the GPU with ID=0. You can provide a comma separated list if you want to use several GPUs (e.g, -g 0, 1). If you want to run it on CPU only, use -g -1, although it will be 20 to 100 times slower.

NOTE: There used to be a bug in Tensorflow multi-GPU inference that caused the program to crash if the batch_size was not divisible by the number of GPUs. We

implemented a patch to solve this issue in DeepEMhancer v 0.16. If your program crashes at the last iteration and you see a strange error message, chances are you are using an older version. In that case, update to the newer version or use only one GPU.

- **Batch size (-b):** Number of map chunks to process simultaneously. Reduce it if CUDA Out of Memory Error happens and increase it if low GPU performance is observed.

DeepEMhancer chunks the map into overlapping cubes of size 64x64x64 with a stride of 8 voxels. The default batch size is 8 cubes. This number has been optimized to work well with 8 GB GPUs.

- **Deep Learning models path (--deepLearningModelPath):** By default, DeepEMhancer models are downloaded at `/home/sanchezg/.local/share/deepEMhancerModels/production_checkpoints/` and this path is automatically assumed. If you have downloaded your models somewhere else, use `--deepLearningModelPath` argument to provide their location (`production_checkpoints` directory).

Alternatively, you can edit the DeepEMhancer config file to point to your directory. The config file is located at:
`${YOUR_DEEPEMHANCER_ENVIRONDIR}/lib/{python3.XX}/site-packages/deepEMhancer/config.py`

Then change the value of the following variable
`DEFAULT_MODEL_DIR`

5. GUI

cryoSPARC:

CryoSPARC includes a DeepEMhancer wrapper that allows you to launch it directly in their GUI. In order to execute it, you need first to have DeepEMhancer installed, and you need to find where the executable and the models are located.

To find the executable, just activate the DeepEMhancer environment and use `which`:

```
conda activate deepEMhancer_env
which deepemhancer
```

Then, go to the Project Level in the GUI and configure the “Path to DeepEMhancer Executable” as a Project-level Parameter.

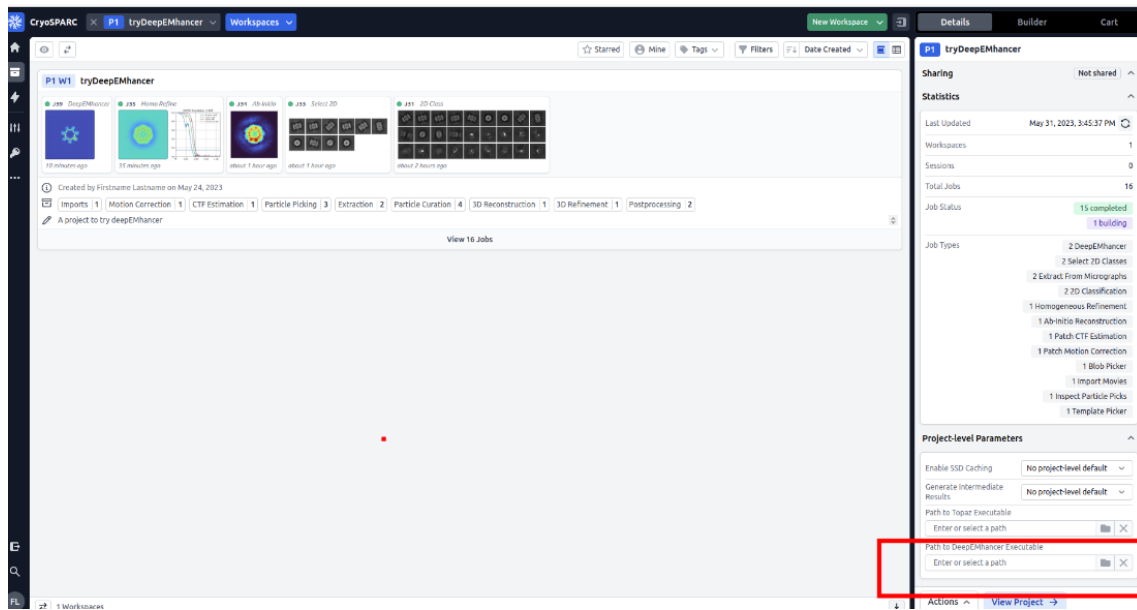


Figure 5. cryoSPARC project viewer. You can set the Path to DeepEMhancer Executable variable here.

Alternatively, for older cryoSPARC versions you will need to provide the executable path each time you launch a job (orange box). In addition, if your DeepEMhancer models are not at the default location (`$HOME/.local/share/deepEMhancerModels/production_checkpoints`), you also need to provide this path as part of the GUI form.

New Job ▾ Details Builder Active Cart

sted ▾

P1 J41 DeepEMhancer

Details ▾

Inputs 0 connected ▴

Input volume volume 0
Min: 0, Max: 1, Repeats: no

Input mask mask 0
Min: 0, Max: 1, Repeats: no

You do not want to use a mask!

Parameters 0 custom, 11 total ▴

All Default Custom ☒ Show advanced

DeepEMhancer settings 0 / 11

Path to deepEMhancer executable
Not set

Path to deepEMhancer models
Not set

Model name
tightTarget ▾

Use half maps
☒

Filter to resolution (Å)
Not set

Normalization mode
0: Automatic ▾

Noise mean
Not set

Noise standard deviation
Not set

Cleaning strength

Figure 6. CryoSPARC DeepEMhancer job form. Input map should be dragged into the red box. You DO NOT want to provide a mask. If the Path to DeepEMhancer Executable project variable was not set, you need to provide it in the form, at the orange box.

All the additional options are exactly the same as described before. Please notice that the current implementation of the wrapper will only run on one GPU, the `CUDA_VISIBLE_DEVICE=0`.

Now is your turn, create a project, import a volume and execute DeepEMhancer.

NOTE: You should import the raw map. If only half maps are available, please, compute the average map. You can do it in Chimera using the following commands:

```
vop add #1,2
vop scale #3 factor 0.5
```

Scipion:

With Scipion, you can try different post-processing methods including DeepEMhancer. If you run the following command

```
scipion tests
xmipp3.tests.test_protocols_deepVolPostprocessing.TestDeepVolPostP
rocessing
```

Scipion will create a project and run the protocols that illustrate how to use DeepEMhancer. Please, note that the example volume is one of very low resolution and that its only purpose is to illustrate how inputs can be provided to DeepEMhancer.

As you can see, the inputs can be provided as a single map or as half-maps. In the case half-maps are desired as inputs, you can feed the protocol with two maps, or with a main with attached half-maps. All the options will lead to the same result and are available only for convenience. The rest of the parameters have been described before in this document.

You can now try to create your own project and import your map(s).

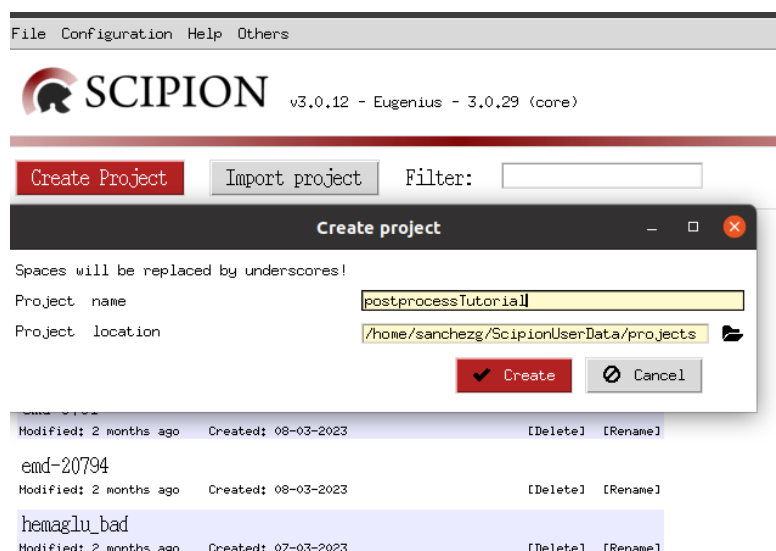


Figure 7. Create a project in Scipion.

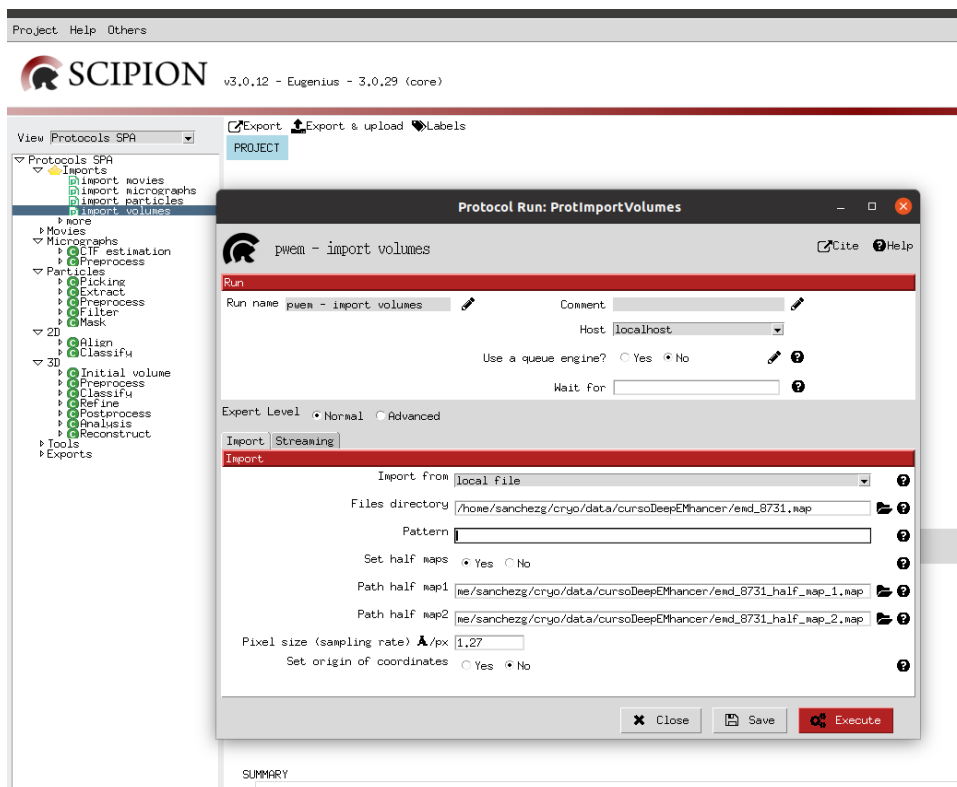


Figure 8. Import volume form in Scipion.

Once you have imported your map(s), you can fill the DeepEMhancer protocol form. You can find it on the 3D>PostProcess menu, or by searching its name after using Ctrl+F. The parameters in the form are exactly the same as presented before. Notice that the GPU ids are to be provided on the FGPU box.

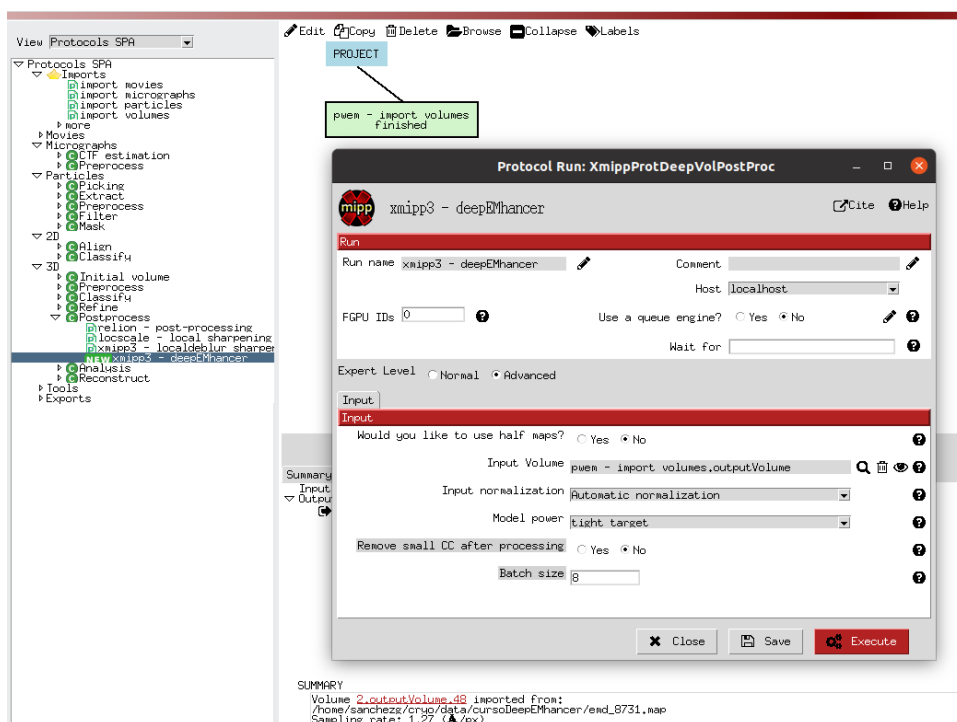


Figure 9. DeepEMhancer protocol form in Scipion.

Spend some time looking at the result. You can visualize it by clicking on Analyze results once the protocol finishes.

Analyze Results

At this point, you may just want to try DeepEMhancer on some other maps, so feel free to launch it while we briefly discuss other post-processing algorithms that you can run from Scipion, namely, LocalDeblur and LocSpiral.

In LocSpiral, maps are enhanced by locally normalizing the amplitudes of the map comparing them to the noise distribution. For those reasons, LocSpiral requires a loose mask to separate the protein from the noise. It does not matter if the 1s in the mask contains noise, what matters is that there is no protein in the 0s.

LocalDeblur performs a local correction that is proportional to the local resolution of the maps. In particular, LocalDeblur, “unblurs” the voxels using a gaussian kernel that is proportional to the local resolution. Consequently, the only required input for LocalDeblur, in addition to the map, is a local resolution map, that we recommend computing using MonoRes.

A full description of how to use LocalDeblur can be found at the Scipion Model Building Tutorial chapter 7.

<https://scipion-em.github.io/docs/release-3.0.0/docs/user/tutorials/modelBuilding/050-volumeScenario.html>.

MonoRes requires a binary mask prior execution, that can easily be computed using the protocol xmipp3-create-3d-mask or some others provided by other packages including Relion. Find the protocol form using Ctrl+F and the keyword “mask”.

The image shows the 'xmipp3 - create 3d mask' protocol form in Scipion. The form is divided into two main sections: 'Mask generation' and 'Postprocessing'. The 'Mask generation' section is currently active, showing fields for 'Mask source' (Volume), 'Input volume' (pwm - import volumes,outputVolume), 'Operation' (Threshold), and 'Threshold' (0.02). The 'Postprocessing' section is also visible, showing options for 'Remove small objects', 'Keep largest component', 'Symmetrize mask', 'Apply morphological operation' (dilation), 'Structural element size' (1), 'Invert the mask', and 'Smooth borders'. The form includes a 'Run' section at the top with 'Run name', 'Comment', 'Host', and 'Wait for' fields. At the bottom, there are 'Close', 'Save', and 'Execute' buttons.

Figure 10. Xmipp protocol form to generate 3D masks in Scipion.

Once the binary mask has been generated, you can run the MonoRes protocol.

Protocol Run: XmippProtMonoRes

Run name: xmipp3 - local MonoRes

Host: localhost

Parallel Threads: 4

Use a queue engine? ☐ Yes ☒ No

Wait for:

Expert Level: ☐ Normal ☒ Advanced

Input

Would you like to use half volumes? ☒ Yes ☐ No

Are the half volumes stored with the input volume? ☒ Yes ☐ No

Input Half Maps: ptem - import volumes,outputVolume

Binary Mask: xmipp3 - create 3d mask,outputMask

Exclude Area:

Extra parameters

Resolution Range (Å): High 0,0 Low 10, Step

Significance: 0,95

Mask threshold: 0,5

Use noise inside protein? ☒ Yes ☐ No

Consider noise gaussian? ☐ Yes ☒ No

Close Save Execute

Figure 11. Monores protocol form. Monores can estimate the local resolution of the maps.

Spend some time looking at the local resolution map. You can visualize it by clicking on Analyse results once the protocol finish.

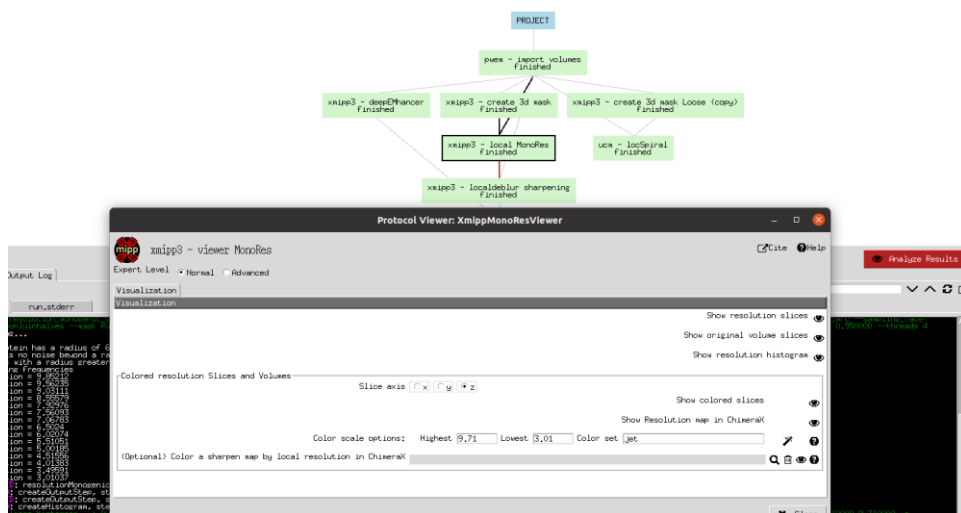


Figure 12. Monores Analyze results viewer.

Now that you have computed a local resolution map, you can finally execute LocalDeblur. LocalDeblur requires as input the map to be sharpened and the local resolution map. The advanced parameter Lambda is the regularization constant, and its default value tends to work well most of the times. If the optimization gets stuck, you may try to reduce it.

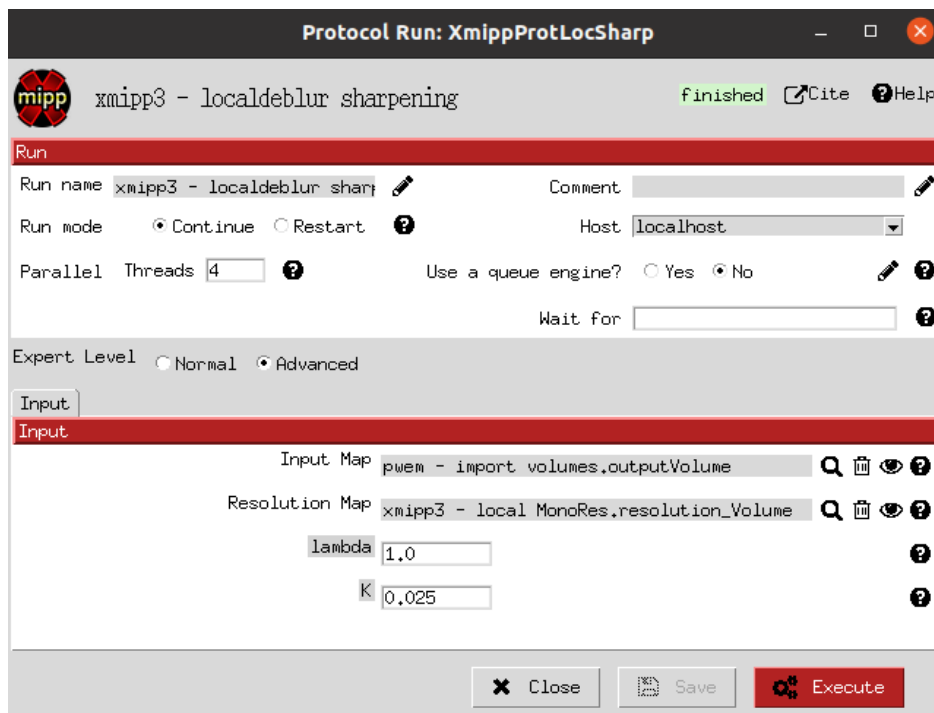


Figure 13. LocalDeblur form in Scipion.

Do not forget to check the results by clicking on Analyse Results.

You can also try to run LocSpiral, another post-processing algorithm. In this case, the most important parameter is the maximum resolution of the map, that we tend to use the global FSC resolution.

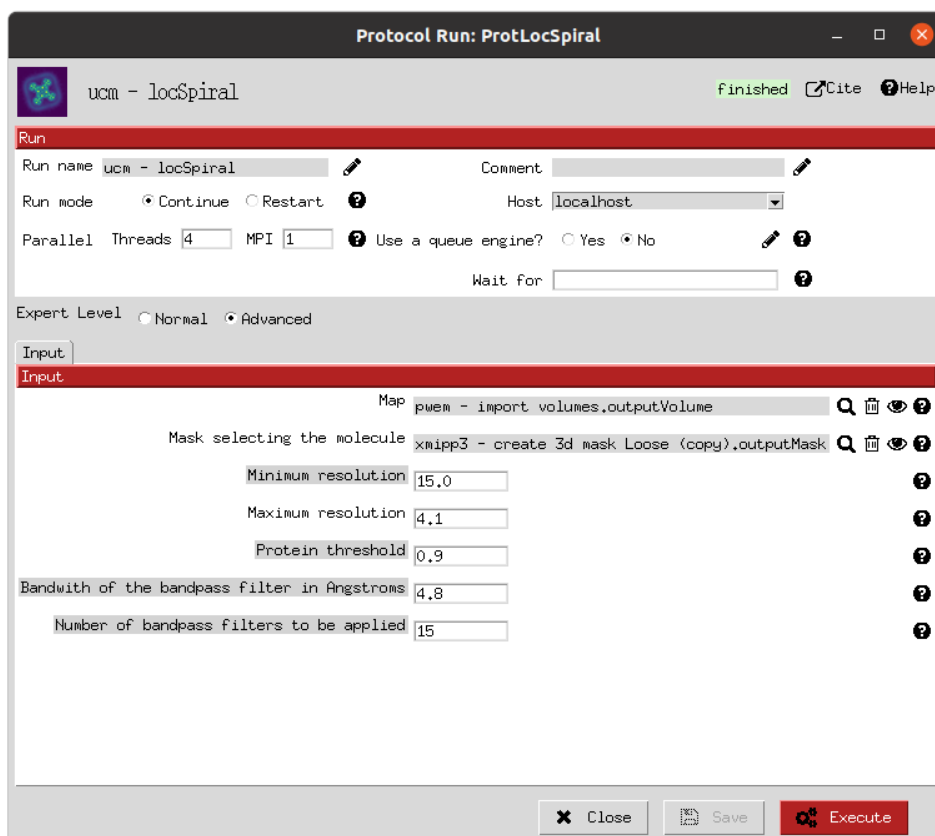


Figure 14. LocSpiral form in Scipion.

Spend some time analysing the results. How do they compare to DeepEMhancer results? Our best advice is to open several maps together in Coot so that you can get the atomic model.

If we would like to compare several of those post-processed maps, for instance, by taking the map difference, it would be better if you re-scale them before. You can use the the “xmipp – volumes adjust” protocol for that.

6. Anisotropy attenuation

It has been observed that map post-processing methods in general, and deepEMHancer in particular, are capable of attenuating map anisotropy despite not being explicitly designed for that purpose. On the other hand, the software package splsoNet has been designed explicitly to attenuate anisotropy. In both cases, the correction essentially involves filling in missing information, which, while effective, carries risks.

In this part of the tutorial, you will work with the Influenza hemagglutinin (HA) trimer reconstructed from tilted images (EMD-8731), that exhibits moderate amounts of anisotropy. You will be running deepEMhancer and [splsoNet](#) on it. Please refer to spisoNet GitHub page for a detailed description of installation and advanced usage details: <https://github.com/IsoNet-cryoET/splsoNet>.

We can run deepEMhancer as before

```
conda activate deepEMhancer_env #Activate the environment

deepemhancer -i emd_8731_half_map_1.map -i2 emd_8731_half_map_2.map -o
emd_8731_demh.mrc
```

In order to run [splsoNet](#), we need first to compute the 3DFSC, that determines the directional resolution of the reconstruction. It is also advisable to use a mask.

```
conda deactivate #To deactivate the deepEMhancer environment

conda activate spisonet #Activate the environment

spisonet.py fsc3d emd_8731_half_map_1.map emd_8731_half_map_2.map
emd_8731_msk_1.map --ncpus 8 --limit_res 3.5
```

This will generate a file called FSC3D.mrc, which is required to launch spisonet reconstruct

After that, we can run the spisonet reconstruction algorithm

```
spisonet.py reconstruct emd_8731_half_map_1.map --i2
emd_8731_half_map_2.map --aniso_file FSC3D.mrc --mask emd_8731_msk_1.map
--alpha 1 --limit_res 3.5 --epochs 20 --output_dir isonet_maps
```

This will generate two corrected half-maps. The level of detail of the maps depends on the value of alpha. Larger values of alpha lead to smoother results. If time permits, try also alpha 0.5 and compare results.

In addition, splsoNet can be B-factor sharpened to increase the level of detail.

```
relion_postprocess --i corrected_emd_8731_half_map_1.mrc --i2
corrected_emd_8731_half_map_2.mrc --o postprocess --auto_bfac --
autob_lowres 10
```

7. LocScale2

In the last part of the tutorial, we will run the different executions modes of LocScale2. For more details, please, read the documentation at the following link:

<https://locscale.readthedocs.io/en/latest>

For the model-based and hybrid map sharpening modes, LocScale2 requires a working instance of REFMAC5. If you already have CCP4/CCPEM installed, check if the path to refmac5 is available in your environment. For model-free sharpening and confidence-aware density modification, REFMAC5 is not required.

LocScale2 offers several execution modes, each serving different purposes:

1. **Model-based LocScale** - The classical method using full atomic model information to locally rescale amplitudes

```
locscale -em path/to/emmap.mrc -mc path/to/model.pdb -res 3 -v -o  
model_based_locscale.mrc
```

You can provide as input the two half-maps instead with `-hm halfmap1.mrc halfmap2.mrc`. `-res` is the resolution for REFMAC5 refinement. You can skip refinement with `--skip_refine`.

2. **Hybrid LocScale** – Like the classical mode, but it automatically builds a pseudo-atomic model for the parts of the map for which an atomic model is not available.

```
locscale -em path/to/emmap.mrc -mc path/to/partial_model.pdb -res 3 -o  
hybrid_locscale.mrc --complete_model
```

This mode is better when preserving contextual information (like membranes) and when you have a partial model. For symmetric structures, add the `-sym` flag (e.g., `-sym D2` for D2 symmetry).

3. **Model-free LocScale** - Performs automated local sharpening without requiring any atomic model information. It uses a neural network to predict a reference map and then applies classical Model-based LocScale on it.

```
locscale -em path/to/emmap.mrc -o model_free_locscale.mrc
```

This is the quickest method for testing local sharpening effects and is useful when no model information is available. This does not modify phases, only amplitudes. As a consequence, the enhancement is not as noticeable as in deep learning-based methods, but it can be regarded as safer.

4. **LocScale-FEM (Feature-Enhanced Maps)**: This is similar to deepEMhancer. One of the main differences is that deepEMhancer removes membranes, but LocScale-FEM does not. It produces a post-processed map in which both phases and amplitudes have been changed.

```
locscale feature_enhance -em path/to/emmap.mrc -gpu 1 -o  
feature_enhanced_prediction.mrc
```

This mode produces a feature-enhanced map along with a confidence map. The confidence map can be used to detect which parts of the enhanced map are not present in an amplitude modified-only map and the other way around.

In order to colour the map based on the confidence map, use the following command

```
color sample #1 map #2 palette -95, #0000ff; -80, #00ffff; 0, 00ff00; 80, #ffff00; 95, #ff0000
```

Now, let's apply different LocScale2 modes to the maps we've been working with:

1. First, try the feature-enhance mode on EMD-7099:

```
locscale feature_enhance -hm emd_7099_half_map_1.map  
emd_7099_half_map_2.map -v -gpu 1 -o emd_7099_locscale_fem.mrc
```

2. Next, use the model-free approach on EMD-7099:

```
locscale -hm emd_7099_half_map_1.map emd_7099_half_map_2.map -o  
emd_7099_locscale_mf.mrc
```

3. Finally try the model-based approach:

The following will use the reference model alone

```
locscale -hm emd_7099_half_map_1.map emd_7099_half_map_2.map -mc 6BHU.cif -  
res 3.5 -v -o emd_7099_locscale_mb.mrc
```

While the following will try to complete the reference model using a pseudo-atomic model.

```
locscale -hm emd_7099_half_map_1.map emd_7099_half_map_2.map -mc 6BHU.cif -  
res 3.5 -v -o emd_7099_locscale_mb2.mrc --complete_model
```

Open all the post-processed maps in ChimeraX to compare:

- Original map
- DeepEMhancer output
- LocScale2-FEM output
- LocScale2 model-free output (if applicable)
- LocScale2 model-based output (if applicable)

Focus on:

1. How each method handles the lipid signals in EMD-7099
2. The interpretability of side chains and backbone in all maps
3. The missing loop that connects the transmembrane domain with the outside domain.